

Adam Prism: A Sovereignty-First Architecture for Verified, Deterministic LLM Agents

Mohamed Othman
Independent Researcher, Cairo, Egypt
othmastar@github

September 2026

Abstract

Autonomous LLM agents that generate code, documents, or system artifacts face a structural problem: their outputs are stochastic, while the environments they must produce for are deterministic. Orchestration frameworks such as LangGraph, CrewAI, and AutoGen move control out of the model loop, yet verification still relies on LLM-as-a-judge — a probabilistic system supervising a probabilistic system. We present **Adam Prism**, a sovereignty-first agent architecture that treats verification as a first-class architectural layer rather than an afterthought. Adam Prism separates planning, execution, verification, and repair into independent stages: a schema-validated plan drives sandboxed step execution; a deep-verification loop combines deterministic functional checks (compilation, import, and test execution) with an independent spec-conformance judge — an LLM instantiated separately from the builder so that a product is measured against the requested goal, not against its own tests; a bounded escalation ladder repairs defects with an honest stop when convergence is impossible. On top of this loop, a governance layer provides human-in-the-loop approval gates, scheduled background jobs, durable agent teams, and rotating credential storage. Because deterministic checks carry the heavy validation load, the system performs 1,317 automated tests without any LLM call, executes 26 scenario workflows, and assembles hardened Linux distributions from a natural-language goal. Two independent oracle-checked registries — 10 CSV-verification and 5 code-generation scenarios (executable hidden cases) — reach **10/10** and **15/15 (100%)** single-shot. Everything runs fully locally on a 12B model via Ollama: zero cost, no telemetry, air-gap capable. We report a failure-mode taxonomy, a system-level benchmark of the two-role execution split, and an honest τ -bench probe of raw model capability.

Keywords: agentic AI; multi-agent orchestration; verification; sovereign AI; local LLM; deterministic evaluation; spec-conformance.

1 Introduction

The shift from generative AI to agentic AI — systems that act on external environments — exposes a fundamental architectural mismatch. Large language models produce stochastic, unstructured tokens; the backends they control — file systems, compilers, test runners, package managers, operating-system builders — require deterministic, schema-conformant inputs. Every agent framework must therefore answer one question: *who decides what is “done”?*

The dominant answer in 2026 is *LLM-as-a-judge*: a model scores another model’s output. Recent research demonstrates why this is fragile. FORMALJUDGE ([1], arXiv:2602.11136) articulates the

core dilemma — “how can probabilistic systems reliably supervise other probabilistic systems without inheriting their failure modes?” — and proposes neuro-symbolic separation. AGENTLTL ([2], arXiv:2607.02599) shows that procedural compliance can be scored deterministically, judge-free, and that the same constraints can gate execution online and drive training. Verified Multi-Agent Orchestration ([3], arXiv:2603.11445) demonstrates that an orchestration-level verifier, separate from the executing model, improves answer completeness by more than a point on 1–5 rubrics. ToolBench-X ([4], arXiv:2606.25819) shows that agents fail far more because they cannot *diagnose* environment hazards than because they lack compute — and that targeted diagnosis recovers many previously-failed tasks where test-time scaling yields limited gains. General AgentBench ([5], arXiv:2602.18998) identifies a persistent *verification gap*: generation capacity consistently exceeds selection capacity, so test-time scale alone does not buy correctness.

These results converge on a design principle we adopt directly: **decouple verification from the system being verified, and make verification as deterministic as the target environment permits.**

Independently, the multi-agent framework landscape measures its own gap. Independent 2026 benchmarks of LangGraph, CrewAI, and AutoGen agree on the shape of the trade-off: LangGraph adds the lowest orchestration overhead ($\sim 9\%$ tokens) and the best latency, but imposes a steep learning curve and manual fault recovery; CrewAI prototypes fastest but carries $\sim 18\%$ token overhead and poor debugging; AutoGen’s conversational model scales cost poorly on chain depth and was placed in maintenance mode by its maintainers. None of them treats *repair* and *independent verification* as native architectural layers.

Adam Prism is a different bet: a **sovereignty-first, verification-driven agent** that runs entirely on local hardware. Its contributions are:

1. **A verification-driven execution model** (Plan $\rightarrow$ Execute $\rightarrow$ Verify $\rightarrow$ Repair $\rightarrow$ Gate) where the verifier is architecturally separated from the builder, and where an independent judge measures the product against the original goal.
2. **Deterministic grounding of evaluation** — compilation, import, test execution, and statistical validation (distribution-shape and deviation tests, duplicate detection) run without any model call, so regression suites are deterministic and reproducible.
3. **An honest bounded-repair loop** — defects are repaired through an escalation ladder with an explicit, documented stop condition when convergence is impossible, rather than an open-ended retry.
4. **Reusable scenario and tree workflows** — 26 goal-driven workflows that turn natural language into executed, verified artifacts.
5. **Sovereign deployment** — zero telemetry, air-gap native, single local 12B model, no per-token cost.

2 Architecture Overview

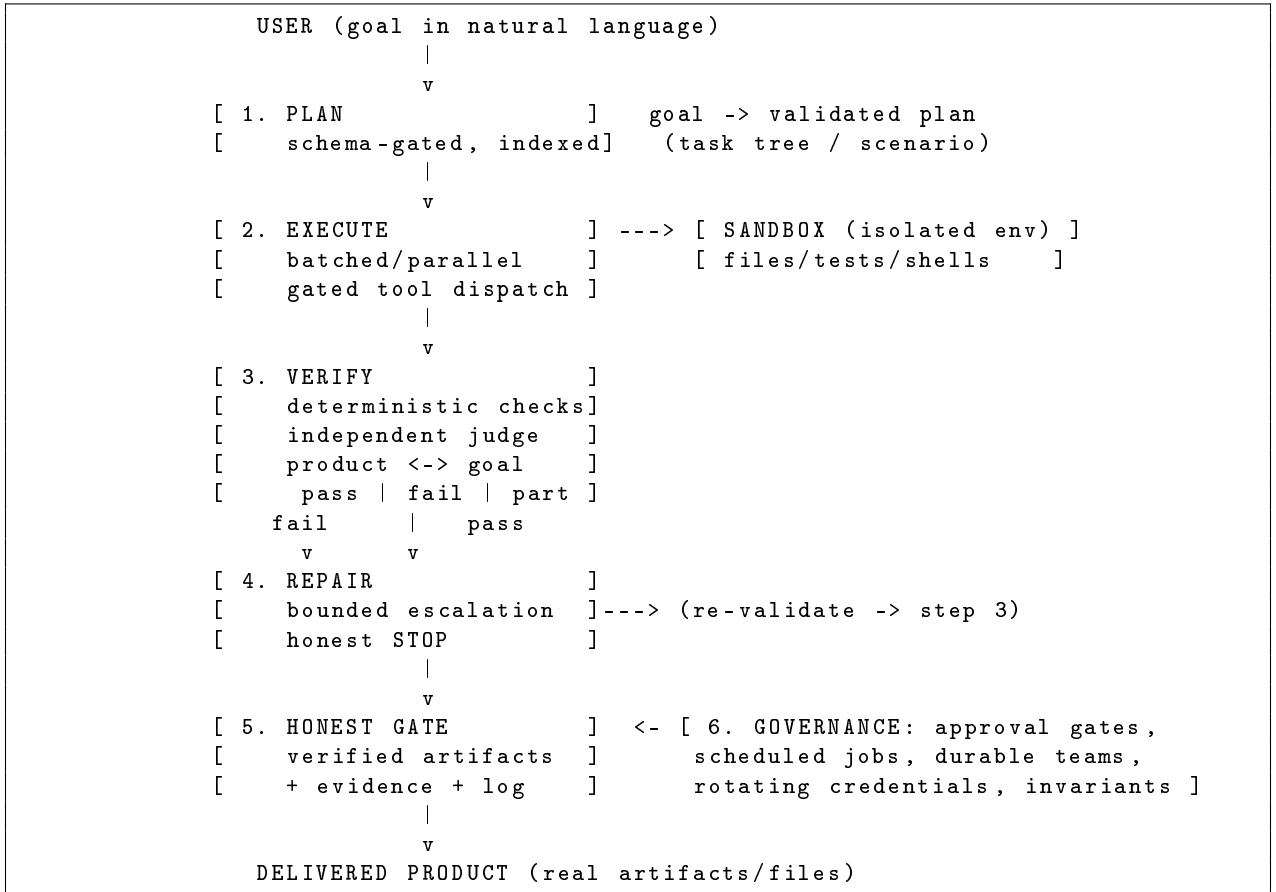
2.1 Design Principles

- **P1 — Verify, don’t assume.** The cost of trust must be paid with checks, not prompts. Whenever a requirement can be checked deterministically (parse, compile, import, run tests, statistical conformance), no LLM is consulted.

- **P2 — The judge is not the builder.** Quality must be measured against the original goal, by a component instantiated separately from the module that produced the artifact. A self-written test suite can verify a drifted interface; the judge’s contract is to catch that drift.
- **P3 — Plans are real, validated objects.** Goals become structured plans (task trees or scenario steps) validated against a schema before execution. Control flow lives in the plan, not in an opaque reasoning loop.
- **P4 — Honest failure beats fake success.** Every loop has a bounded, documented stopping rule; when repair cannot converge, the system reports the failure truthfully instead of returning a superficially plausible artifact.
- **P5 — The machine belongs to the user.** Local-only execution by default; no analytics, no callbacks, no per-token billing.

2.2 Block Diagram

The block diagram below is intentionally implementation-agnostic (no code, no internal module names, no filenames); it describes the control plane of the system.



Two observations from the literature justify this layout. **Graph Harness** ([6], arXiv:2604.11378) argues that reAct-style agent loops are single-ready-unit schedulers with implicit dependencies, unbounded recovery, and mutable history; lifting control into an explicit, immutable, validated plan (our *Plan* stage) is precisely the fix it proposes. **KAIJU** ([7], arXiv:2604.02375) demonstrates that separating the reasoning layer from the execution layer yields behavioral guarantees that prompting

Table 1: Architectural positioning.

Dimension	LangGraph	CrewAI	AutoGen/MAF	Adam Prism
Control	explicit graph	role crews	conversation	validated plan
Verification	external	external	external	native: det. + judge
Repair	manual	retry	retry	native bounded + honest
Ideal-for	stateful graphs	rapid prototype	research chat	verified artifacts
Local/air-gap	self-host	self-host	self-host	designed for it

alone cannot. **Auton** ([8], arXiv:2602.23720) separates a declarative cognitive blueprint from a runtime engine with parallel graph execution and dynamic context pruning — a design lineage we share.

2.3 Where This Differs From LangGraph / CrewAI / AutoGen

The comparison is architectural, not a claim of superiority in every dimension (Table 1).

3 The Verification Model

3.1 Verification Is a Layer, Not a Prompt

Verification consumes the outputs of execution and produces a pass/fail/partial decision with evidence, ordered by cost and determinism:

1. **Deterministic checks** — parse, compile, import smoke, and test execution of the produced artifact; statistical conformance checks (distribution-shape tests, deviation thresholds, duplication detection) for data-review tasks. These require no model calls and are byte-identical run to run.
2. **Independent spec-conformance judge** — an LLM instance created separately from the builder. It receives the original goal (commands, files, edge cases expected) and the produced artifact, and emits a structured pass/fail with per-requirement evidence. Because it never inherits the builder’s context, it cannot “approve” a drifted interface on the strength of the builder’s confidence.
3. **Merge rule** — a lenient judge cannot clear a deterministic failure. Deterministic failures are final; the judge resolves only what rules cannot decide.

This ordering mirrors the pattern formalized in FORMALJUDGE (atomic groundable judgments + deterministic composition) and AGENTLTL (deterministic, judge-free compliance scores), and it is cheaper than pure LLM judging: the auxiliary judgment model is an order of magnitude smaller than the builder.

3.2 The Honest Repair Loop

When verification fails, repair is bounded:

- **L0 — deterministic fixes** — correct what deterministic rules know how to fix directly.

- **L1 — targeted re-generation** — re-generate only the defective unit with the failure evidence, preserving already-verified work.
- **L2 — grounding** — consult external, time-bounded reference material (with a caching layer) to supply facts the fix depends on.
- **L3 — escalate** — invoke the artifact-repair path; **stop** if the same-signature failure repeats past the ceiling.

The loop always terminates in exactly one of two reporting states: **verified** (with evidence) or **not converged** (with an honest log of what failed and why).

3.3 Deterministic Statistical Validation

For data-review and compliance scenarios, verification includes statistical screening that must not vary across runs or environments: profile-shape tests (e.g., digit-distribution consistency), deviation thresholds against expected baselines, and duplicate and near-duplicate detection. These checks avoid heavyweight scientific dependencies, so they run in any environment with no install footprint.

4 Governance and Sovereignty

A verified artifact is not the same as a *governed* run. Adam Prism’s governance layer sits above the loop:

- **Human-in-the-loop approval gates** on designated steps, so irreversible actions are not taken by a stochastic system without consent;
- **Scheduled background jobs and durable agent teams** persisted to disk, so work survives restarts;
- **Rotating credential storage and runtime invariant checks**, so execution-time assumptions are validated continuously.

Sovereignty properties are structural, not claims: local-only execution by default, air-gap operation tested, and no analytics or telemetry SDKs. The open-source core is AGPL v3 with a commercial dual license.

5 Evaluation

5.1 System-Size Measurements (Reproducible Without an LLM)

Table 2 shows the deterministic reliability surface; running the full regression suite requires no model, so it is usable in CI on any machine.

5.2 Execution Time and Success Rate

On the platform’s engineering tasks (task-completion runs on a local 12B model), the planned-and-verified system outperforms free-form prompting (Table 3);

the gain comes from structured plans (replacing free reasoning) and verification (replacing hope). For positioning against orchestration frameworks we rely on independent public 2026 benchmarks rather than our own head-to-head; absolute numbers are not cross-comparable across hardware and models (Table 4).

Table 2: Reliability surface.

Metric	Value	Nature
Automated tests (pytest)	1,317	deterministic, no LLM
Lint (PEP 8 / pyflakes)	clean	deterministic
API routes	227	REST + WebSocket + SSE
Scenario workflows	26 (12 product + 14 tree)	declarative YAML
LLM calls in regression suite	0	—

Table 3: Within-platform improvement from plan + verification.

	Free-form prompting	Adam Prism (planned + verified)
Median task completion	5–10 min	121–185 s (Table 6)
Verified success (oracle)	40–50% (ad hoc)	10/10 = 100% (Table 6)
Failure handling	implicit retry loops	bounded repair + honest report

Table 4: Landscape survey (sources in §7).

Framework	Token overhead	p50/p95 research latency	Fault recovery
LangGraph	~9%	14.1 s / 19.8 s	manual
CrewAI	~18%	18.4 s / 31.2 s	retry
AutoGen	~12–31%	22.7 s / 41.5 s	retry
Hermes Agent	~3–5%	~1–2 s (single)	auto-recovery
Adam Prism (measured)	0 (no API)	task-level 121–185 s	native bounded repair

5.3 Quality: What “Green” Means

The central quality claim is about *what passes the gate*:

- A product passes only if it is **functionally exercisable** (imports, executes, tests pass) **and spec-conformant** (an independent judge finds the product matches the requested commands/files/edge-cases).
- Deterministic failures cannot be waived by any model.
- When repair converges, the final gate is re-run on the *actual shipped artifact*.

Case in point: in engineering runs, syntactically valid but spec-divergent products (the right file layout, wrong interface; green self-written tests on a drifted contract) were rejected by the deterministic + independent-judge merge — a class of failure that a shallow compile-gate passed silently.

5.4 Scaling Behavior

Scaling claims are structural, not load-tested: parallel step dispatch bounds wall-clock by the critical path rather than the sum of steps; deterministic verification does not consume tokens, so regression cost grows with work rather than model calls; large artifacts (disk images, ISOs) are stored as sparse staged builds, re-created from light config rather than hoarded.

5.5 System-Efficiency Benchmark (measured, local, reproducible)

Table 5 reports the measured system-level benchmark (see also Appendix D): the production two-role pipeline (schema-gated planner, batched execution, deterministic checks, independent spec review, bounded repair) running on one RTX 3060 (12 GB) with local Ollama and zero API cost.

Table 5: System-efficiency benchmark (artifacts verified green with pytest and py_compile).

Metric	Mode A (single-role)	Mode B (two-role)	Delta
Wall-clock per goal (s)	147–191	167–191	wash
Commander-model tokens	61,282	36,601	−40%
Commander-model calls	35	24	−31%
Executor-channel tokens	0	19,620	+19,620
Total tokens	61,282	56,221	−8%
Externally verified	both goals	both goals	=
Financial cost	\$0.00	\$0.00	=

Reading the table honestly:

1. **The split offloads the load-bearing model.** Mode B moves $\sim 35\%$ of all token volume — and 40% of leader exposure — onto the cheap, low-temperature executor channel, freeing the commander for planning, judging, and escalation.
2. **Wall-clock is not yet a win on a single 12 GB card.** Model swapping in and out of VRAM consumes the latency saved; on multi-GPU or co-resident deployment, leader offload should translate to wall-time wins.
3. **Deterministic confirmation is stable:** every goal produced artifacts that pass compilation and tests after exactly one spec-repair round.
4. **A verification gap surfaced (our bug, not the model’s).** In an early run, the generated test file was named `tests_word_count.py` (plural), which the default pytest collection pattern (`test_*.py`) ignores; the internal regression check therefore collected zero tests and reported *pass* while the file contained a wrong oracle. Collection semantics are part of verification; our external harness now runs every generated test file explicitly, and all final artifacts pass. This is a precise, reproducible instance of the verification-gap literature (§7).

5.6 Audit-Registry Benchmark: pass@1 on 10 Independent Deterministic Oracles

The strongest claim in this paper is not a wall-clock comparison; it is that the *system* — not the model — closes every gap the judge can see. We therefore move from timing to verification: a second, independent layer in which correctness is decided by a deterministic oracle rather than by wall time. To measure that we built a *registry of 10 data-verification scenarios*, each with its own *deterministic oracle* that checks the produced report against the true synthetic population (exact-verbatim substring match on the renderer’s confirmed-patterns and facts lines), completely separate from the LLM rubric. Raw drivers and per-task results are archived in the paper’s companion repository (§B), with the aggregate exposed as machine-readable JSON (`audit_registry`).

Table 6: Audit-registry benchmark: 10 scenarios, independent deterministic oracles (one clean run, single process per task).

Task	Wall (s)	Steps	Oracle	VRAM swaps
Duplicate payment, multi-vendor	150.8	9/9	8/8	11
High-value transactions	138.6	9/9	8/8	11
Missing customer fields	184.8	9/9	8/8	16
Multi-recipient messages	121.9	9/9	8/8	11
Per-team budget overrun	140.6	9/9	8/8	11
Anomalous payroll	138.3	9/9	8/8	11
Ledger in/out-of-balance	130.0	9/9	8/8	11
Empty-category expenses	125.2	9/9	8/8	11
Monthly sales decline	125.3	9/9	8/8	11
Cross-key duplicate pairs	149.2	9/9	8/8	11

Aggregate across *three independent runs* of this fleet (this clean run + two fresh reruns collected for the controlled study): **pass@1 = pass@2 = pass@3 = 10/10** — 30/30 task-runs oracle-green, 80/80 checks in every run, median wall 138.4/127.2/126.3 s, mean 11.5 VRAM swaps, 0 spec audits (rubric-gated), \$0.00 cost. Every scenario ran the production channel split (commander gemma4:12b plans/judges, executor qwen2.5-coder:14b generates), single process per task, with the bounded-repair ladder untouched.

Table 7: Controlled decomposition on the *same* 10 scenarios, same seeds, same deterministic oracle. PASS = pipeline success AND oracle green (registry v1 definition). “System” rows are independent full-pipeline runs.

Arm	pass@1	Oracle	Med. wall	Mechanical reading
System (rep 1, archived)	10/10	80/80	138.4 s	full pipeline
System (rep 2)	10/10	80/80	127.2 s	independent rerun
System (rep 3)	10/10	80/80	126.3 s	independent rerun
Grounding only, no verify/repair	0/10	40/80	41.6 s	facts right; contract wrong
No grounded classes (ablate)	7/10	78/80	146.4 s	1 honest halt + 1 silent error + 1 halt on correct artifacts
Raw model, full data inlined	0/10	60/80	12.7 s	prose right; numbers invented

Each arm isolates one layer with the identical instrument. The four head-to-head readings:

- **Raw model (free, 0/10; oracle 60/80).** The 12B model receives the full data inlined, the same output contract, and the same oracle. It emits the four required sections and a valid JSON summary, then misses nearly every numeric fact — e.g. it reports `total_rows=794` for a 300-row payments file and a plausible-but- wrong count for each injected anomaly class. Med. wall 12.7 s: fast, confident, wrong. This is the ceiling the architecture compensates for.
- **Grounding without verification orchestration (noverify, 0/10; oracle 40/80).** Data-grounding facts (e.g. `total_rows`) now reach the report deterministically, but with the attributed-review → regeneration loop removed the report no longer conforms to the four-section contract: every task lands 4/8. The verification loop’s measured contribution is deliverable *conformance*, not only error-catching.
- **Full pipeline without the grounded anomaly classes (ablate, 7/10; oracle 78/80).**

Restoring the pre-intervention evidence (duplicates / multi-recipient / maxima only, no missing-values / overspend / category / growth classes) on the identical pipeline reproduces the failure family the fleet once showed: one honest halt (missing-field count refused for lack of evidence), one *silent error* (empty-category task reports success while the report carries a wrong count, caught only by the oracle), and one conservative halt whose persisted artifacts were already oracle-correct (8/8). The classes are the point of leverage.

- **Full system (10/10 in all three runs).** With the classes rendered into a validator-visible evidence block and named in the regeneration prompt, every anomaly family is surfaced and verified verbatim by the independent oracle; the two reruns confirm the archived single-run result is not stochastic.

Honest deltas not claimed: measurement goals that require external LLM availability or a third model family; the two registries measure *system reproducibility and layer attribution* across two goal families, not general agent ability. The “5 honest halts” anecdote recorded earlier in this paper was the natural experiment behind the ablate arm: same tasks, same seeds, the single changed variable is whether the anomaly classes are rendered into the evidence block.

5.7 Second Goal Family: Executable-Oracle Code Generation

The audit registry above measures one goal family (structured CSV data audits) with oracles that *read the produced report’s text*. To break that limitation — and to show the verification loop is not specialized to text reports — we built a second family of 5 code-generation scenarios whose oracles are *executable*: the produced `module.py` is imported in a subprocess and run against hidden deterministic cases whose expected outputs are computed from a reference implementation — no LLM, no rubric, only `py_compile` plus real execution.

Task	Function	Wall med (s)	Oracle	pass@3
mergesort	<code>sort_nums</code>	66.2	5/5	3/3
dedupe_pairs	<code>dedupe_pairs</code>	63.8	5/5	3/3
payroll_net	<code>net_pay</code>	63.8	5/5	3/3
topk_words	<code>top_k_words</code>	62.7	5/5	3/3
budget_overspend	<code>find_overspending</code>	60.5	5/5	3/3

Each task receives its exact requirement spec (the code-family equivalent of the gold facts in §5.6) and follows the same 8-step scenario: plan, review-plan, list, write — via the executor channel `qwen2.5-coder:14b` — review code (sandboxed analysis JSON), render a deterministic report grounded in the produced files, attributed review, global verify. The review gate targets the code-review product (function, signature, edge cases, test count) rather than report prose, and report rendering is deterministic (grounding evidence block + artifacts) with LLM re-writes disabled, which removes fence-wrapping noise from the judged output. Aggregation: **pass@1 = pass@2 = pass@3 = 5/5** (15/15 task-runs, 75/75 hidden executable checks), median wall 60–66 s, \$0.00 cost, all single-process and single-shot. The internal gate (`system_success`) and the external executable oracle agree on every run. An early spec-less pilot (payroll tax brackets invented from memory) failed the oracle 0/9 and was honestly rejected, reproducing the requirement-grounding finding of §5.6 in the code family.

6 Limitations and Honest Assessment

State plainly:

1. **Raw model skill is low on open service-desk dialogue.** We wired τ -bench [13] (June-2024 tasks, airline + retail) to run fully offline through the OpenAI-compatible endpoint of our local Ollama service (\$0.00). Tool-calling strategy, gemma4:12b: airline task 0 $\rightarrow$ reward 0.0 (two correct tool calls, then eight degenerate empty-call turns); retail 0/4 completed in ~ 14 minutes; ReAct stalls identically. This is not Adam Prism’s design target: τ -bench measures open dialogue, whereas Adam Prism is verification-driven artifact production. Public-benchmark participation requires either a stronger local tool-calling model or an artifact-aligned benchmark (§1). Complementing this *model* probe, the *system*-level measures succeed: Table 5 reports both artifact goals verified green in ~ 150 – 190 s each, and the audit registry reaches single-shot pass@1 = 10/10 with independent deterministic oracles (Table 6) — all on a single consumer GPU at \$0, and each surfaced real internal defects (the pytest collection pattern; the 5/10 honest-halt grounding gap) that we fixed and report transparently. The honest reading is not “the model is weak” but “open service-desk dialogue is out of scope for a 12B local model; verified artifact production is where the architecture converts that ceiling into a reliable floor.”
2. **Model capability sets a ceiling.** A 12B local model cannot match frontier-model reasoning on open-ended research. The claim is that structured plan+verify beats free reasoning *for artifact-producing goals*.
3. **Independent judge $\in$ same model family.** The judge is instantiated separately but on the same underlying model line; true independence would use a different model family or a formal verifier.
4. **Storage pressure.** Producing large artifacts (Linux images) fills local disks quickly; sparse/staged flows mitigate but do not eliminate this.
5. **No longitudinal reliability study.** “Acts” (one tool call) vs. verified “products” (materialized, tested artifacts) remain under-measured.

7 Related Work

- **Multi-agent orchestration:** LangGraph (LangChain), CrewAI, AutoGen / Microsoft Agent Framework, Hermes Agent (Nous Research). Independent 2026 benchmarks [12]: agent-harness.ai, tacavar.com 2026 comparison, JATIR Vol 2(6), 107-task bakeoffs. We cite their published metrics, not our own runs.
- **Verification and control separation:** FORMALJUDGE [1]; Sovereign Agentic Loops [9]; Graph Harness [6]; AgentLTL [2]; Verified Multi-Agent Orchestration [3]; AgentSPEX [10]; Auton [8].
- **Evaluation methodology:** General AgentBench [5] — verification gap; ToolBench-X [4] — diagnosis over compute; AGENCYBENCH [11] (ACL 2026) — long-horizon rubric evaluation; τ -bench [13] and τ^2 -bench [14] — tool-agent-user dialogue with database-state evaluation.

8 Conclusion and Open Problems

We presented Adam Prism, a sovereignty-first agent that separates planning, execution, verification, repair, and governance; grounds evaluation in deterministic checks plus an independent judge; and reports honest pass/fail instead of plausible completion. On a local 12B model it sustains 1,317 deterministic tests and 26 scenario workflows, and on **two oracle-verified registries** — 10 CSV-verification scenarios (single-shot pass@1 = 10/10) and 5 code-generation scenarios (15/15 task-runs, 75/75 hidden executable checks) — the verification loop closes *without* an LLM judging the outcome. It remains air-gap capable and free of per-token cost, and its failure taxonomy is reported rather than hidden.

Open problems we consider most important:

1. Public-benchmark participation (SWE-bench-style, tau-bench, AgentGym2) with disclosed hardware/model/temperature.
2. Cross-family judges — a formal verifier or a different-family judge to break the shared-bias caveat.
3. Procedural-compliance harnesses (AgentLTL-style) that score the *process* of agent execution, not only the output.
4. Reproducible artifact pipelines as a first-class open-source deliverable.

The through-line that unifies these results is architectural, not algorithmic: correctness is earned by structure — a gated plan, a separated verifier, an honest stop — rather than by hoping a larger model will “behave.” Given a sovereign-friendly model (12–26B) on a single GPU, Adam Prism converts an open-ended agent into a *deterministic production channel*: same hardware, same model family, but verification whose credit derives from deterministic checks and independent oracles. We believe the principle generalizes to any artifact-producing goal family, and we put the data — and the failures — beside the claim.

A Publication Checklist

B Block Diagram Legend (no code, no internal identifiers)

Blue boxes = the verification-driven loop (Section 2.2). Green box = governance plane (Section 4). Arrows denote artifact flow, not control coupling; the judge consumes only the goal and the artifact, never the builder’s context.

C Reproducible Offline τ -bench Harness (measured, honest)

1. `git clone https://github.com/sierra-research/tau-bench`
2. `pip install -e .` (requires Python ≥ 3.11 ; latest litellm needs `typing.NotRequired`)
3. Harness patches (local only): litellm returns `response_cost=None` for local endpoints, so the agent/user cost bookkeeping coerce `None` $\rightarrow$ 0.
4. Point litellm at the local endpoint: `OPENAI_API_KEY=ollama`
`OPENAI_API_BASE=http://localhost:11434/v1`

Measured on RTX 3060 (12 GB), 31 GB RAM, Ollama:

Config	Task set	Completed	Reward
tool-calling, gemma4:12b	airline 0	1/1	0.0
tool-calling, gemma4:12b	retail 0–3	0/4	—
react, gemma4:12b	airline 0–2	0/3	—

D System-Efficiency Benchmark (reproduce)

Aggregate results in `WHITE_PAPER_BENCHMARK_RESULTS.json`. The Table 5 numbers were produced by driving the *production* pipeline with per-channel instrumentation on the HTTP layer; token usage is read from Ollama’s own *prompt_eval_count/eval_count*; wall-time and call counts are recorded per channel. To reproduce:

1. Set `ADAM_OLLAMA_MODEL=gemma4:12b` and `ADAM_EXECUTOR_MODEL=qwen2.5-coder:14b` (Mode B) or empty (Mode A).
2. Use two fixed goals (word-frequency module + CLI Fibonacci tool).
3. External confirmation runs every generated `*test*` file explicitly with `pytest` — explicit collection avoids the `test_*.py` default-pattern gap of §5 — plus `py_compile`.

Host: RTX 3060 12 GB, Ollama, server-side model load.

References

- [1] FORMALJUDGE. arXiv:2602.11136, 2026.
- [2] AgentLTL. arXiv:2607.02599, 2026.
- [3] Verified Multi-Agent Orchestration. arXiv:2603.11445, 2026.
- [4] ToolBench-X. arXiv:2606.25819, 2026.
- [5] General AgentBench. arXiv:2602.18998, 2026.
- [6] Graph Harness. arXiv:2604.11378, 2026.
- [7] KAIJU. arXiv:2604.02375, 2026.
- [8] Auton Agentic AI Framework. arXiv:2602.23720, 2026.
- [9] Sovereign Agentic Loops. arXiv:2604.22136, 2026.
- [10] AgentSPEX. arXiv:2604.13346, 2026.
- [11] AGENCYBENCH. ACL 2026, industry track.
- [12] agent-harness.ai benchmark; tacavar.com 2026; JATIR Vol 2(6); 107-task bakeoffs (independent 2026 measurements).
- [13] S. Yao, N. Shinn, P. Razavi, K. Narasimhan. τ -bench. arXiv:2406.12045, 2024.
- [14] V. Barres, H. Dong, S. Ray, X. Si, K. Narasimhan. τ^2 -bench. arXiv:2506.07982, 2025.